LABORATORIO DE ROBÓTICA

UR5 con gripper
Control ROS 2 y planificación con MoveIt

Brayan Mateo Bravo Losada
Ingeniería Electrónica
Escuela Colombiana de Ingeniería Julio Garavito
20 de octubre de 2026

Resumen

Se construyeron los paquetes del UR5 con una pinza paralela de dos dedos y se ejecutó una rutina de cuatro puntos mediante tópicos ROS 2. Se generó una configuración con MoveIt Setup Assistant y se verificó la planificación y ejecución de los grupos del brazo y de la pinza desde RViz y desde un nodo Python. Las ejecuciones grabadas completaron ocho movimientos cada una, incluyendo apertura, cierre y confirmación digital simulada.

1. Objetivo y alcance

Integrar descripción cinemática, simulación, controladores y planificación en un workspace local. El brazo y los dedos se simulan en Gazebo. La recogida y entrega de una pieza se representan mediante estados lógicos: esta práctica no acredita agarre por contacto físico ni uso de un robot real.

Componente	Entorno utilizado
Sistema	Windows con WSL2; Ubuntu 24.04.4 LTS
ROS y simulador	ROS 2 Kilted; Gazebo Sim Ionic 9.5.0
Planificación	MoveIt 2 y Setup Assistant 2.14.3; OMPL/RRTConnect
Control	gz_ros2_control; controlador del brazo y JointTrajectoryController de pinza
Base docente	DanielPalominoS/ur5_gripper_packages; revisión 87a547dfde78

Se reutilizó el entorno previamente instalado. Los paquetes se compilaron con colcon; los resultados corresponden a esta implementación y a las mediciones guardadas en los archivos JSON de evidencia.

2. Construcción y rutina por tópicos

La entrega contiene cuatro paquetes: **ur5_robotiq_description**, **ur5_robotiq_bringup**, **lab_manipulador** y **ur5_gripper_moveit**. La compilación concluyó con los cuatro paquetes finalizados. Los lanzadores comparten el dominio ROS 74 y el reloj de Gazebo.

Puntos y operaciones

Punto	Objetivo articular [rad]	Operación
Preparación	[0, -1.57, 1.57, -1.57, -1.57, 0]	Mensaje informativo y apertura
Recogida	[0.35, -1.45, 1.80, -1.92, -1.57, 0]	Cierre y estado lógico de recogida
Intermedio	[0, -1.20, 1.40, -1.77, -1.57, 0]	Salida digital ALTA
Entrega	[-0.40, -1.45, 1.80, -1.92, -1.57, 0]	Apertura y espera digital

Cada objetivo es distinto. Tras la entrega, la aplicación observa una entrada BAJA seguida de una nueva entrada ALTA y regresa a Preparación. El nodo digital permite confirmación manual mediante SetBool o un pulsador automático de demostración que publica el cambio después de 1.5 s.

Interfaz	Tipo / propósito
/scaled_joint_trajectory_controller/ joint_trajectory	trajectory_msgs/msg/JointTrajectory: seis articulaciones del brazo
/gripper_controller/joint_trajectory	trajectory_msgs/msg/JointTrajectory: dos dedos
/joint_states	sensor_msgs/msg/JointState: comprobación de posición final
/lab/digital_input y /lab/digital_output	std_msgs/msg/Bool: señales de entrada y salida
/lab/set_digital_input	std_srvs/srv/SetBool: confirmación manual

Reproducción mínima

Desde la carpeta de entrega: **bash ejecutar.sh build**. Mantener terminales independientes con **bash ejecutar.sh sim true**, **bash ejecutar.sh io true** y **bash ejecutar.sh planificador**. En otra terminal ejecutar **bash ejecutar.sh topics** o **bash ejecutar.sh moveit**, una rutina a la vez. El README incluye comandos manuales completos y el modo de entrada digital manual.

3. Configuración con Setup Assistant

Se cargó **ur_with_gripper_moveit.urdf.xacro** en la interfaz gráfica y se generó la matriz de autocolisiones con 10 000 muestras. El archivo SRDF y los archivos de configuración se produjeron mediante el botón de generación del asistente; la grabación y la captura permiten revisar ese proceso.

Elemento	Configuración
Brazo	Grupo arm; cadena base_link → tool0; solucionador KDL
Pinza	Grupo gripper; gripper_finger_joint y right_finger_joint
Efector	eef; grupo gripper; montaje en tool0; grupo padre arm
Referencia fija	world ya existe en el URDF; no se añadió un virtual joint redundante
Ejecución	FollowJointTrajectory para brazo y pinza

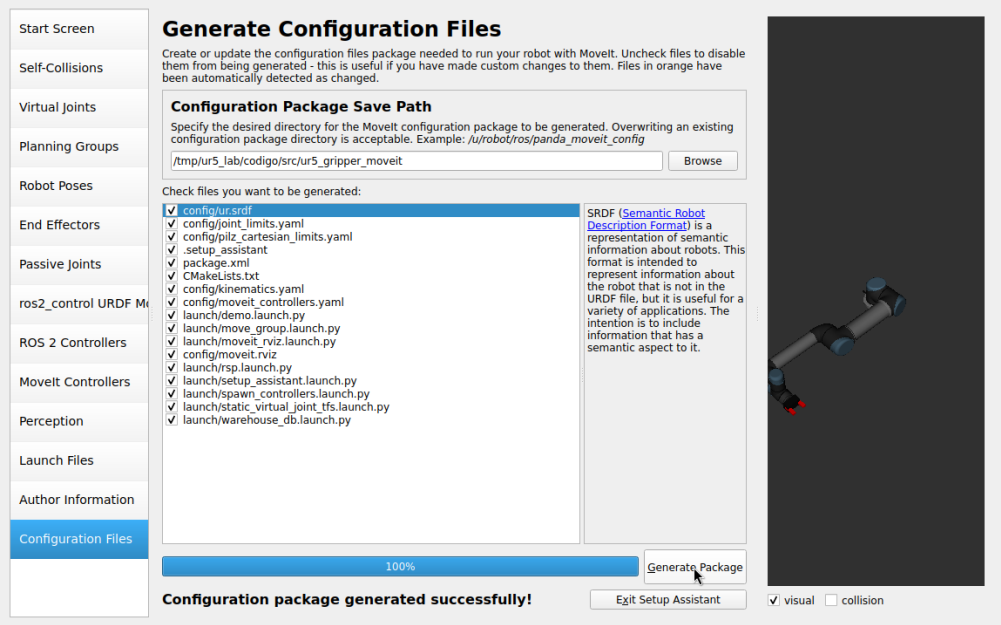


Figura 1. Paquete generado en la interfaz gráfica de MoveIt Setup Assistant.

Ajustes de integración

Después de la generación se alineó el nombre del controlador del brazo con **scaled_joint_trajectory_controller**, se añadieron los cuatro estados del brazo y los estados Open/Close de la pinza, se habilitaron límites de aceleración y se configuró OMPL/RRTConnect. El launch de integración consume los estados y los controladores de Gazebo; no inicia otro gestor de hardware. Estos ajustes están documentados en completar_configuracion.py.

4. Verificación y resultados

Prueba	Movimientos	Error máx. brazo	Error máx. dedo	Tiempo rutina
Tópicos ROS	8/8	0.019704 rad	0.001949 m	70.9 s
Nodo MoveIt	8/8	0.000937 rad	0.000949 m	53.5 s

Los errores se calcularon comparando la consigna con /joint_states. Para el control por tópicos se exigió error menor que 0.02 rad en el brazo y 0.002 m en los dedos durante varias actualizaciones. MoveIt también verificó el resultado de sus acciones y validó ocho planes. Los tiempos son de pared entre el primer y el último evento, no tiempos de simulación.

Pruebas desde RViz

En MotionPlanning se seleccionó el grupo arm, inicio <current> y objetivo Recogida; se pulsó Plan y luego Execute. La planificación tardó aproximadamente 0.079 s y la ejecución terminó con SUCCEEDED. Se repitió el procedimiento con el grupo gripper y el objetivo Close: planificación aproximada de 0.062 s y ejecución correcta. Los tiempos de planificación pertenecen a esas consultas puntuales.

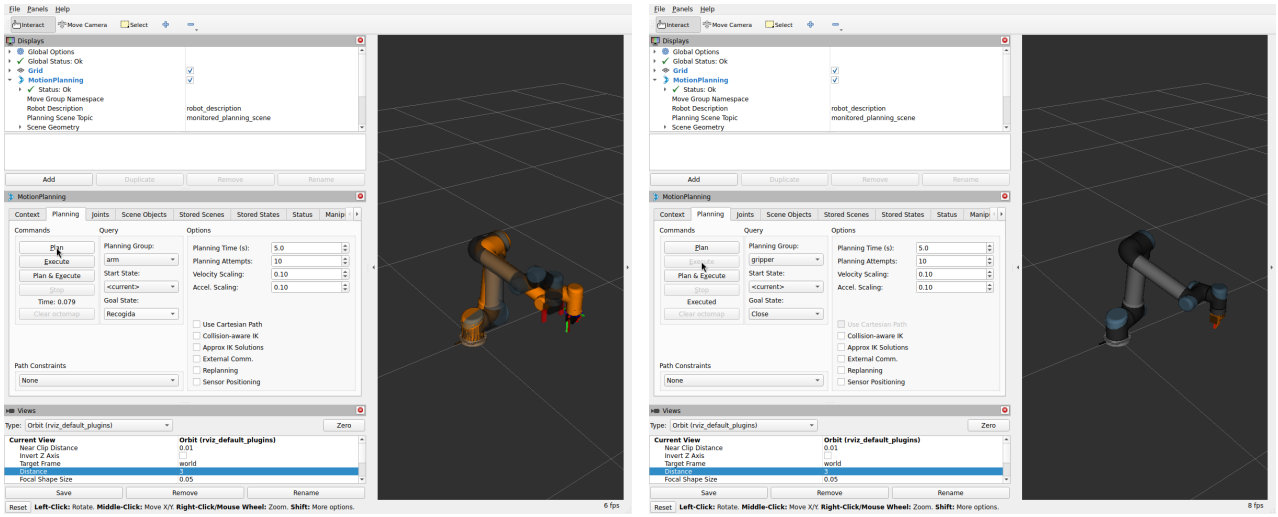


Figura 2. Verificación de los grupos arm y gripper desde el plugin de MoveIt en RViz.

Evidencia auditable

Los JSON contienen objetivos, secuencia de operaciones, muestras articulares y el indicador de éxito. Los videos de las rutinas conservan su tiempo real y muestran los estados de la aplicación. Los resúmenes de Setup Assistant y RViz indican explícitamente reproducción x8 y x4, respectivamente.

5. Hallazgos y recursos de entrega

Hallazgo	Solución aplicada
Control de la pinza	El controlador original recibía posiciones por tópico y no ofrecía GripperCommand. Se usó JointTrajectoryController con los dos dedos y la misma acción FollowJointTrajectory en MoveIt.
Lanzamiento y modelo	Se corrigieron -z, la ruta del mundo, el suelo de espesor cero y los estados iniciales. Las rutas con espacios se entrecomillaron; Setup Assistant se abrió con --urdf_path y un alias temporal.
Planificador instalado	Se utilizó el identificador RRTConnect de los archivos instalados, en lugar del nombre RRTConnectkConfigDefault de la práctica anterior.
Funciones no utilizadas	El escenario no incorpora percepción 3D ni reconocimiento de objetos. Sus avisos de disponibilidad no impidieron las planificaciones y ejecuciones verificadas.

Alcance del resultado

Se verificaron brazo, dedos, señales digitales y planificación en simulación. La pieza es lógica y no se validaron contacto, fuerza de agarre ni transferencia a hardware. La matriz de colisiones está basada en muestreo; no constituye una prueba formal para todas las configuraciones. El trabajo cubre los puntos solicitados de control, nodo de rutina, configuración y ejecución MoveIt, con evidencias de ambas vías.

Enlaces de la solución

Código, configuración, resultados y videos se reúnen en la carpeta del laboratorio del repositorio Mate02010/rospart3. Los enlaces siguientes permiten consultar la solución y descargar las grabaciones. También se adjunta una copia local con la misma estructura de archivos.

Código y guía de ejecución
Nodo Pick and Place
Paquete de configuración MoveIt
Resultados de la prueba mediante MoveIt

Configuración con Setup Assistant (x8)

Planificación de ambos grupos en RViz (x4)

Rutina mediante tópicos ROS, tiempo real

Rutina mediante MoveIt, tiempo real

Referencias

Repositorio y guía del profesor

ROS 2 Kilted: instalación en Ubuntu

Compatibilidad de ROS y Gazebo